SENATI · ESCUELA DE TECNOLOGÍAS DE LA INFORMACIÓN

FULL STACK DEVELOPER SOFTWARE · SEMINARIO

LABORATORIO #11

Autenticación con JWT

Integración de inicio de sesión, sesiones con JSON Web Tokens y autorización para el sistema de inventario.

Estudiante	_____
Docente	Giancarlos Barboza N.
Sección	Seminario IA 501
Fecha	_____

01 · IMPLEMENTACIÓN

Objetivo y flujo de autenticación

El laboratorio integra autenticación basada en JWT entre la interfaz React y la API Express. El acceso a las funciones del inventario depende de una sesión válida; las operaciones de escritura también se someten a autorización por rol en el servidor.

Etapa	Implementación
Inicio de sesión	El cliente envía las credenciales a `POST /api/auth/login`. La API comprueba la cuenta activa y devuelve los datos públicos del usuario junto con un JWT.
Persistencia y restauración	El cliente conserva el token en `localStorage` bajo la clave `accessToken` y consulta `GET /api/auth/me` al restaurar la sesión.
Peticiones protegidas	Un interceptor de Axios adjunta el token como `Authorization: Bearer`. El middleware de Express verifica firma, algoritmo, vencimiento y estado activo del usuario.
Permisos y cierre	El servidor aplica permisos según el rol. Cerrar sesión elimina el token local y la interfaz vuelve al acceso.

Fragmento del cliente: guardar la sesión

```
const { data } = await api.post('/api/auth/login', credentials)
localStorage.setItem('accessToken', data.token)
setUser(data.user)
```

Fragmento del cliente: enviar el token

```
const token = localStorage.getItem('accessToken')
if (token) {
  config.headers.Authorization = `Bearer ${token}`
}
```

Fragmento del servidor: crear un token firmado

```
jwt.sign(
  { rol: usuario.rol },
  obtenerSecretoJwt(),
  {
    algorithm: 'HS256',
    subject: String(usuario.id),
    expiresIn: process.env.JWT_EXPIRES_IN || '2h'
  }
)
```

> **Resultado técnico:** la sesión tiene vencimiento, el endpoint `/me` vuelve a validar el usuario en la base de datos y el backend rechaza tokens ausentes, inválidos o expirados.

Los fragmentos se transcriben de la implementación del proyecto. No se incluyen secretos, contraseñas, hashes ni tokens de sesión.

02 · EJECUCIÓN Y BASE DE DATOS

Servicios locales y persistencia

Las capturas muestran la API y el cliente ejecutándose en el entorno local, junto con la base `bdinventario` y las tablas usadas por el sistema.

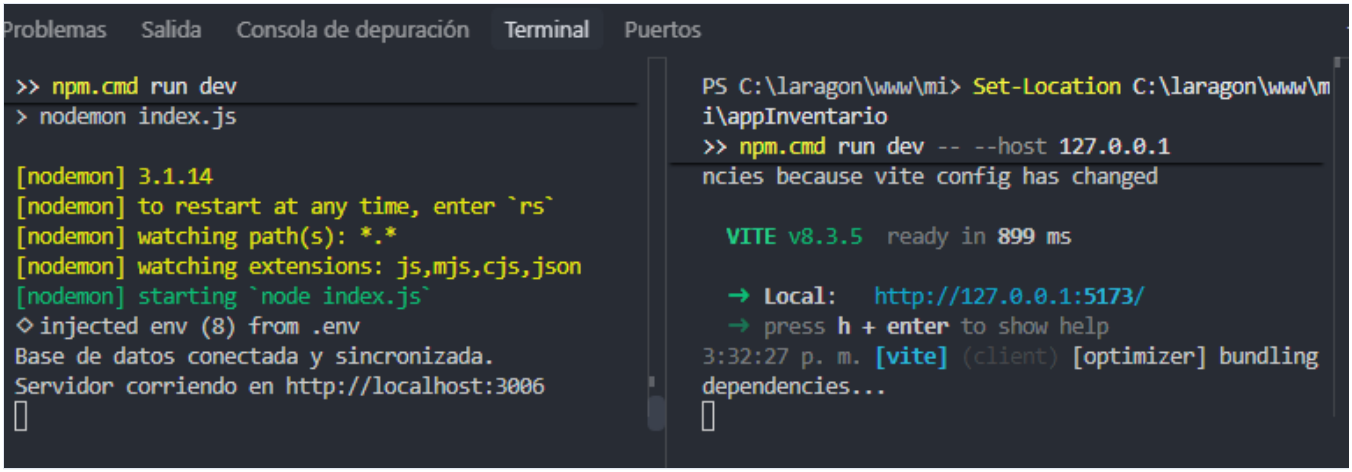


Figura 1. Terminales de ejecución del backend y el frontend.

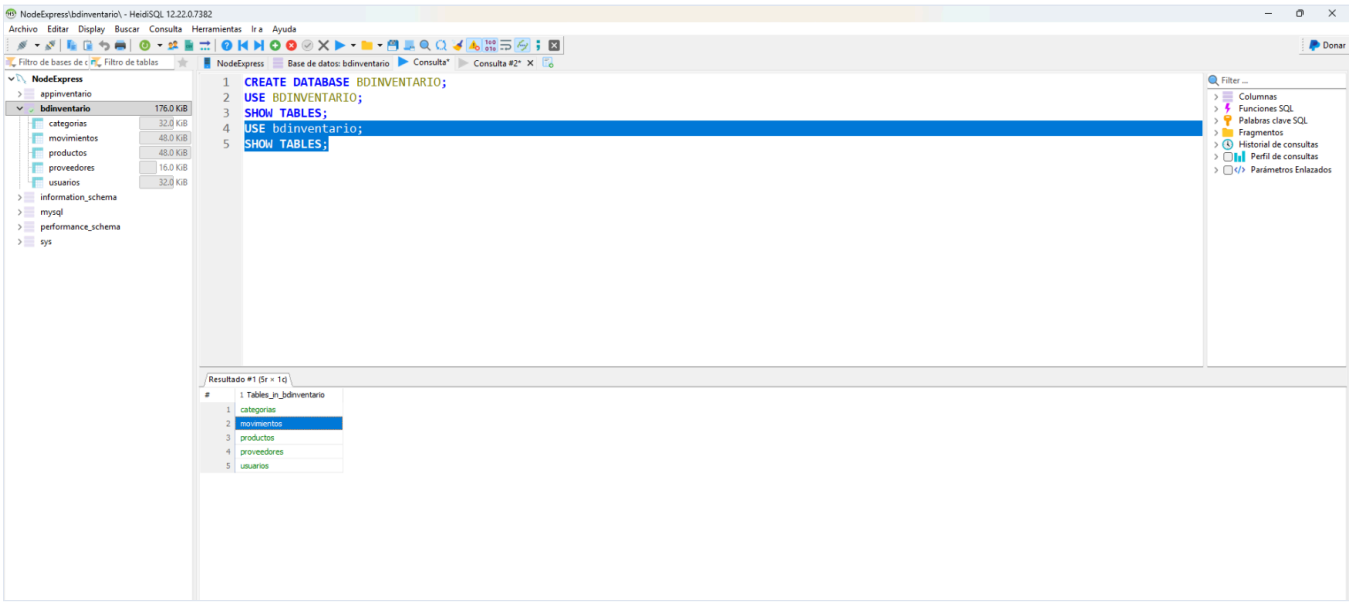


Figura 2. HeidiSQL: tablas de la base de datos `bdinventario`.

> **Persistencia:** los cambios del CRUD se realizan sobre las categorías almacenadas por la API; el listado de tablas confirma la estructura de la base de datos conectada.

03 · SESIÓN Y CRUD EN NAVEGADOR

Acceso autenticado y categorías

La pantalla autenticada identifica la sesión con el rol Administrador y habilita la navegación del panel. En el módulo Categorías se observa el registro de prueba antes y después de su actualización.

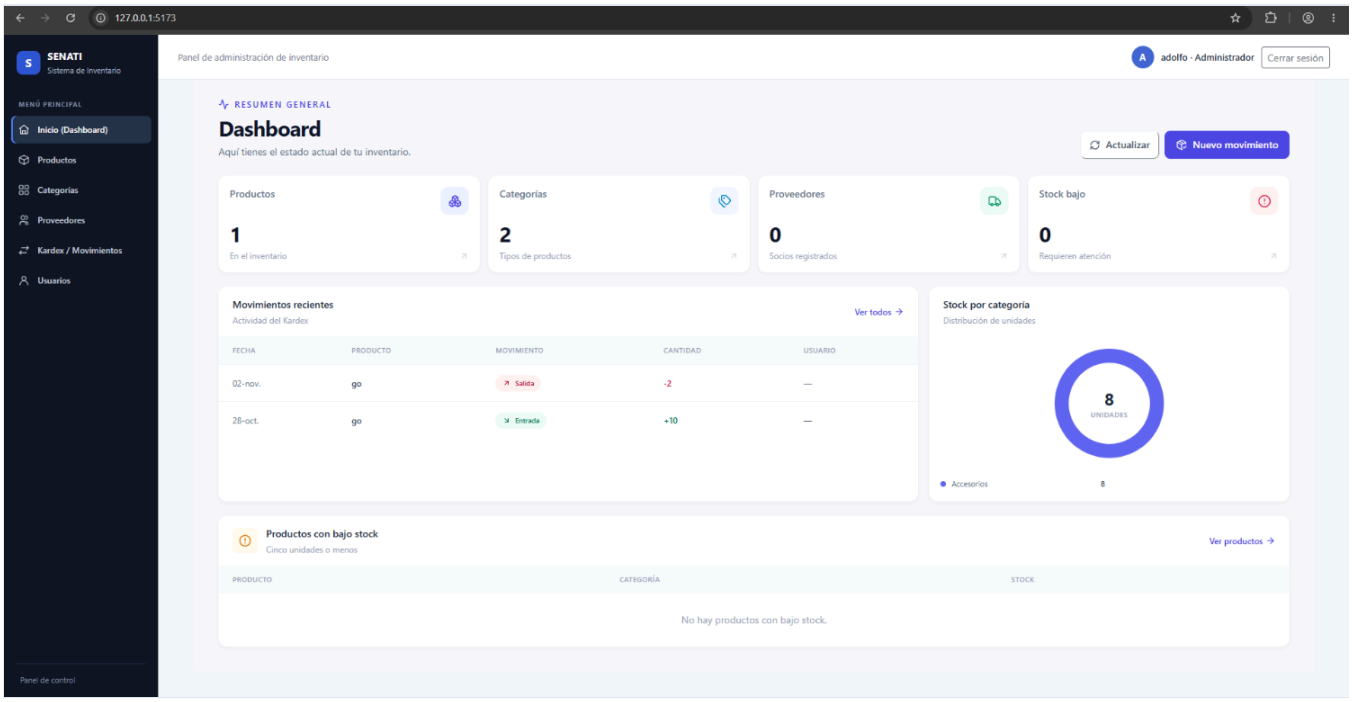


Figura 3. Dashboard con sesión autenticada como Administrador.

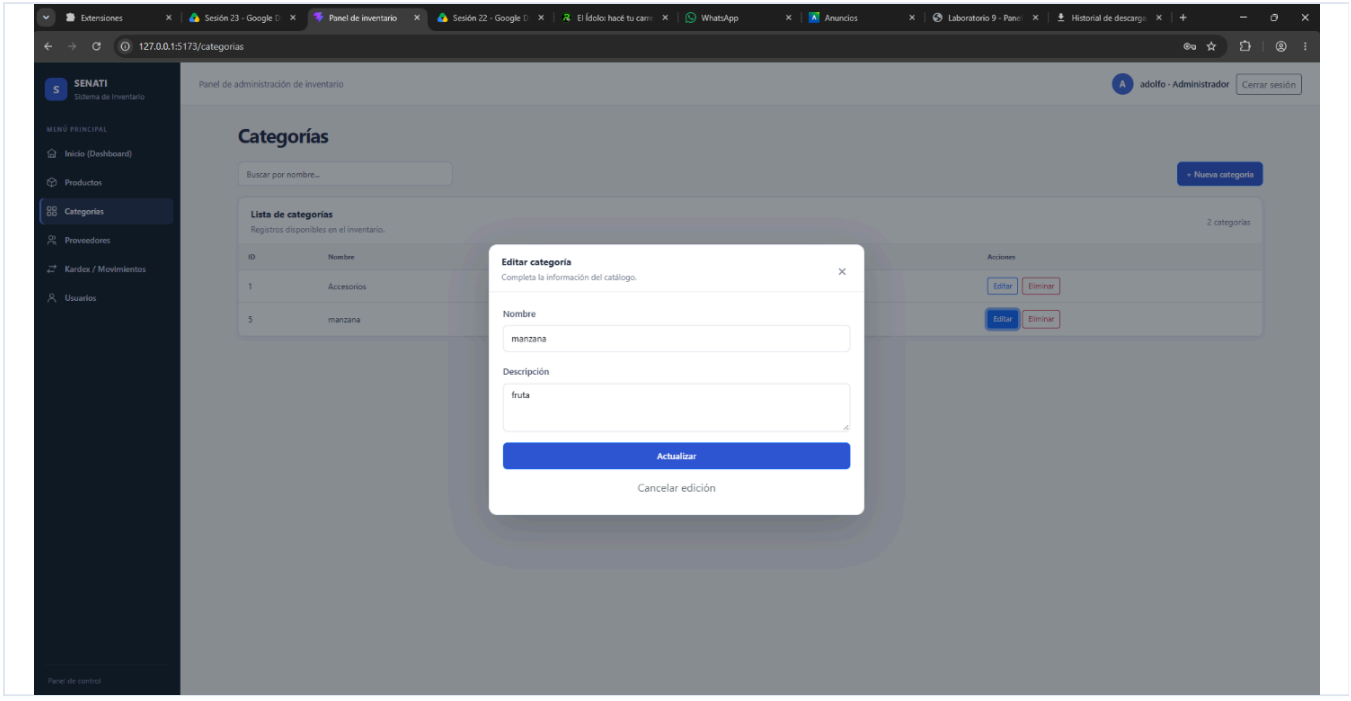


Figura 4. Registro «manzana» presente en el listado y formulario abierto antes de actualizarlo. La creación por API se comprueba con la respuesta POST de la figura 7.

03 · CONTINUACIÓN

Actualización y eliminación

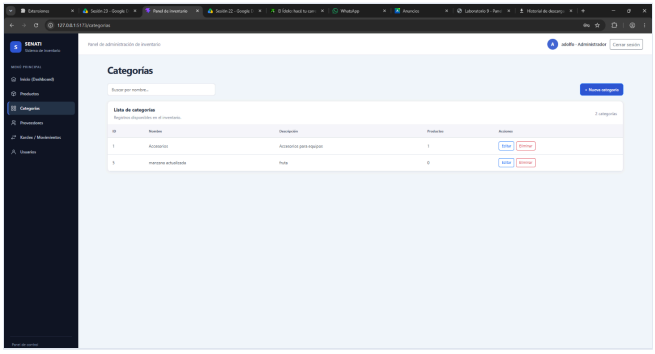


Figura 5. Categoría actualizada en el listado del navegador.

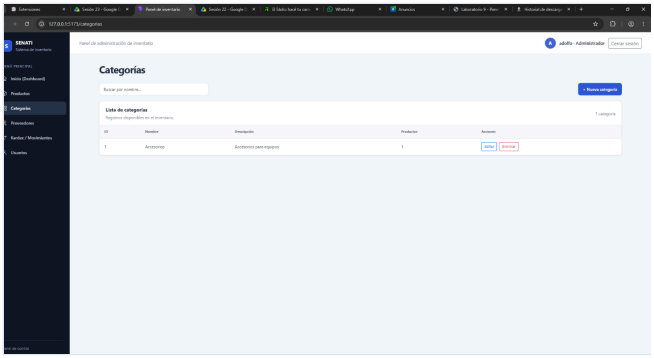


Figura 6. Listado después de eliminar la categoría de prueba.

> **Comprobación visual:** el registro de prueba aparece con el nombre actualizado y deja de aparecer después de la eliminación.

Las capturas se conservan tal como fueron compartidas; se excluyeron las imágenes que mostraban credenciales o tokens.

04 · PRUEBAS DE API

Postman: creación y consulta

Las pruebas de categorías confirman los métodos HTTP y los códigos de respuesta observados. Las operaciones protegidas se ejecutaron con una sesión autorizada.

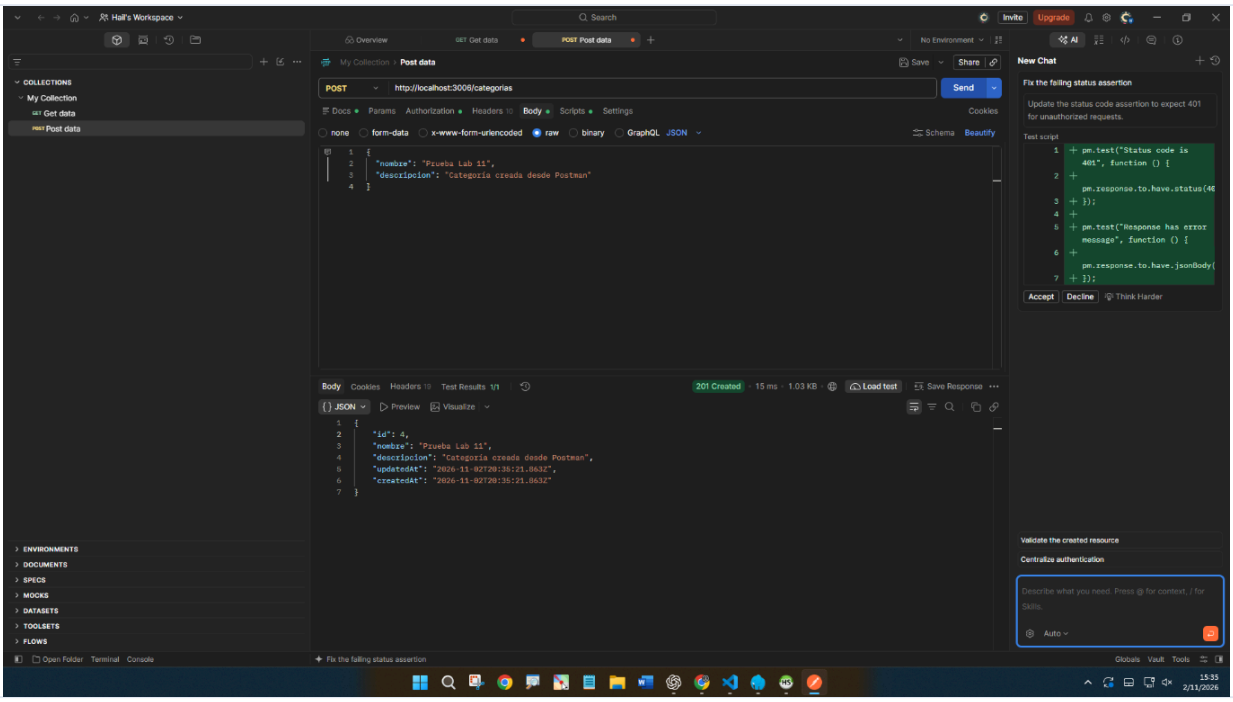


Figura 7. POST: creación de una categoría; respuesta **201 Created**.

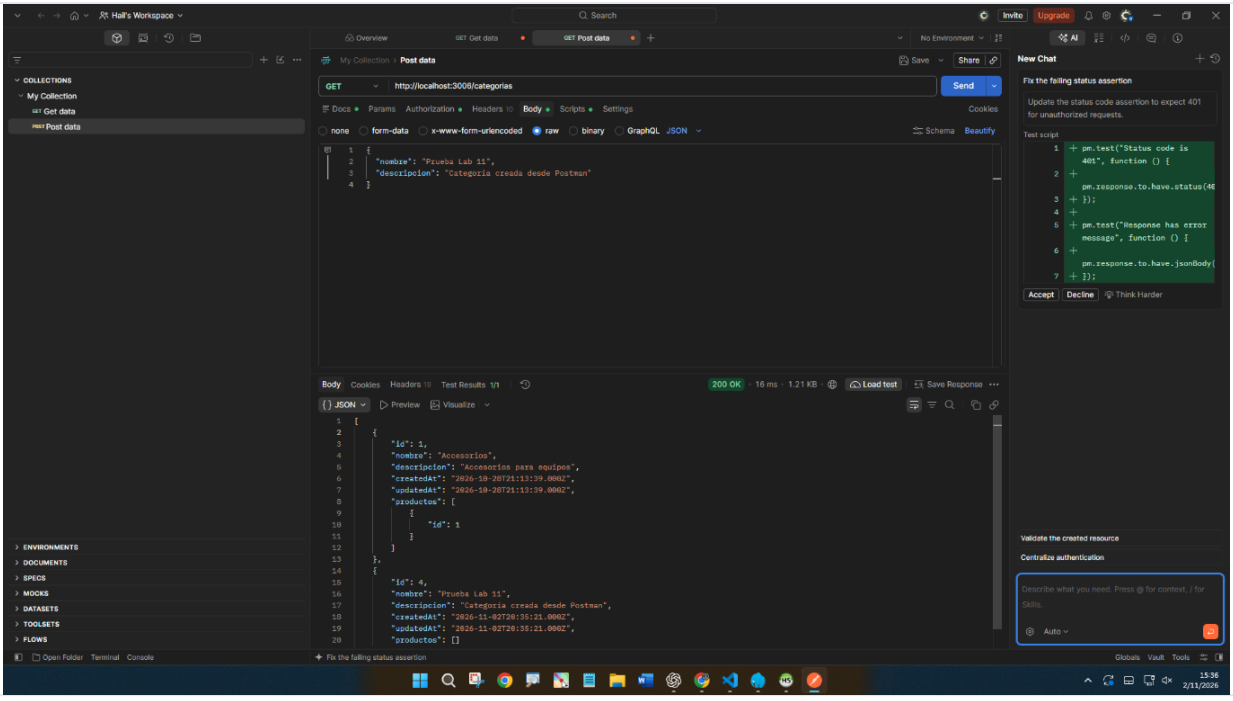


Figura 8. GET: consulta del listado de categorías; respuesta **200 OK**.

04 · CONTINUACIÓN

Postman: modificación y eliminación

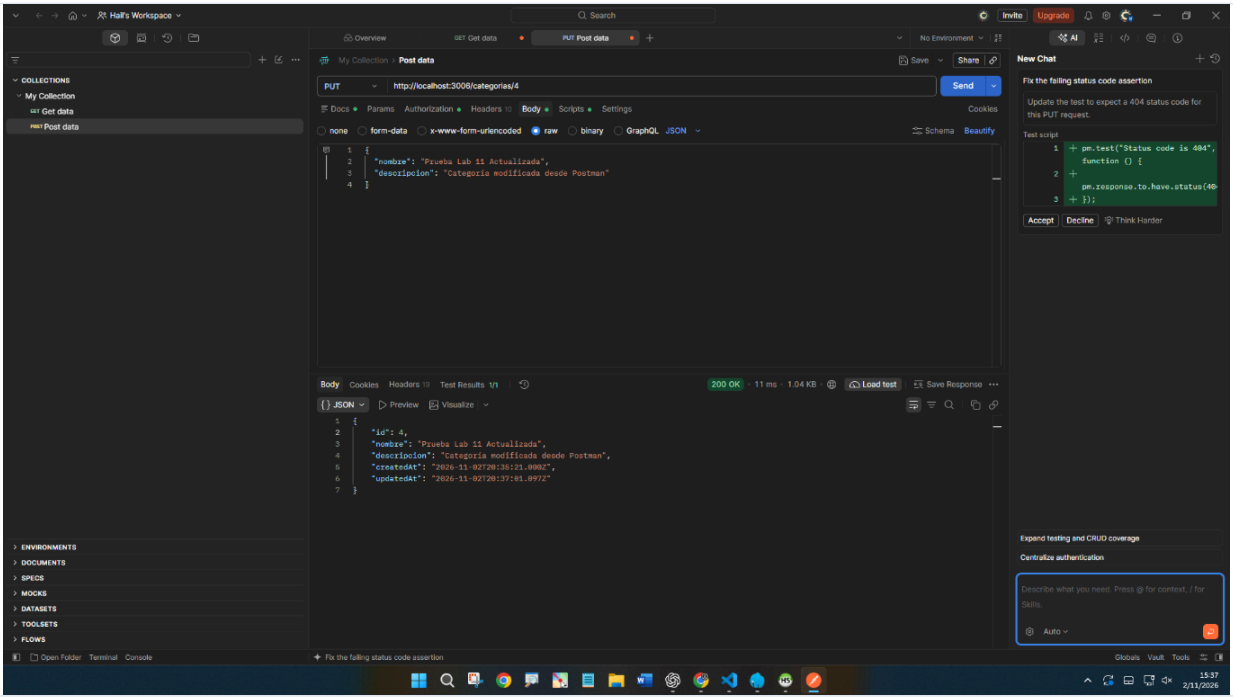


Figura 9. PUT: actualización de una categoría; respuesta **200 OK**.

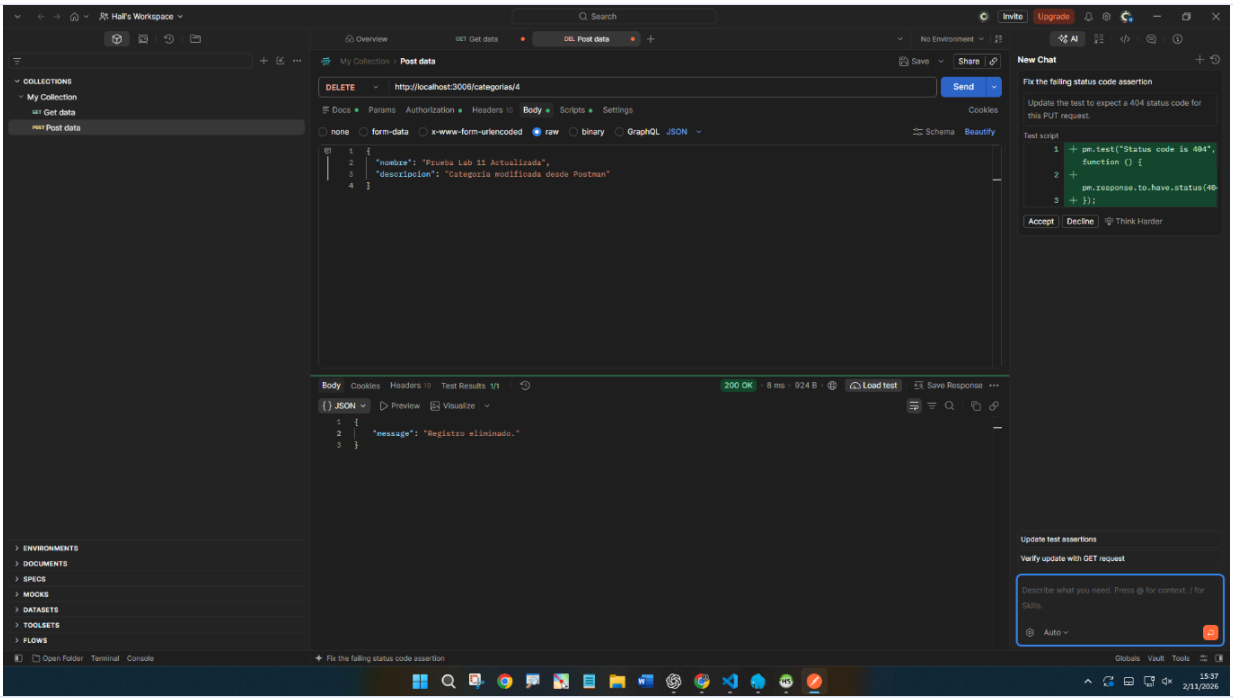


Figura 10. DELETE: eliminación de una categoría; respuesta **200 OK**.

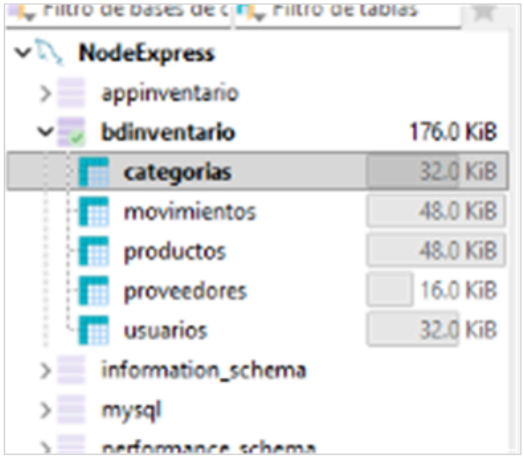
Operación	Resultado	Verificación
POST / categorías	201 Created	Registro creado correctamente.
GET / categorías	200 OK	Listado disponible.
PUT / categorías/:id	200 OK	Registro actualizado.
DELETE / categorías/:id	200 OK	Registro eliminado.

Evidencia de persistencia en MySQL

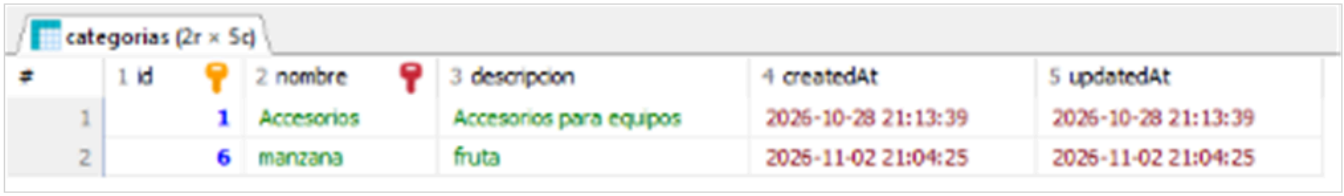
HeidiSQL · Base de datos bdinventario

La tabla categorias contiene el registro «manzana» (id 6), con descripción «fruta».

Base y tabla seleccionada



Resultado de SELECT * FROM categorias



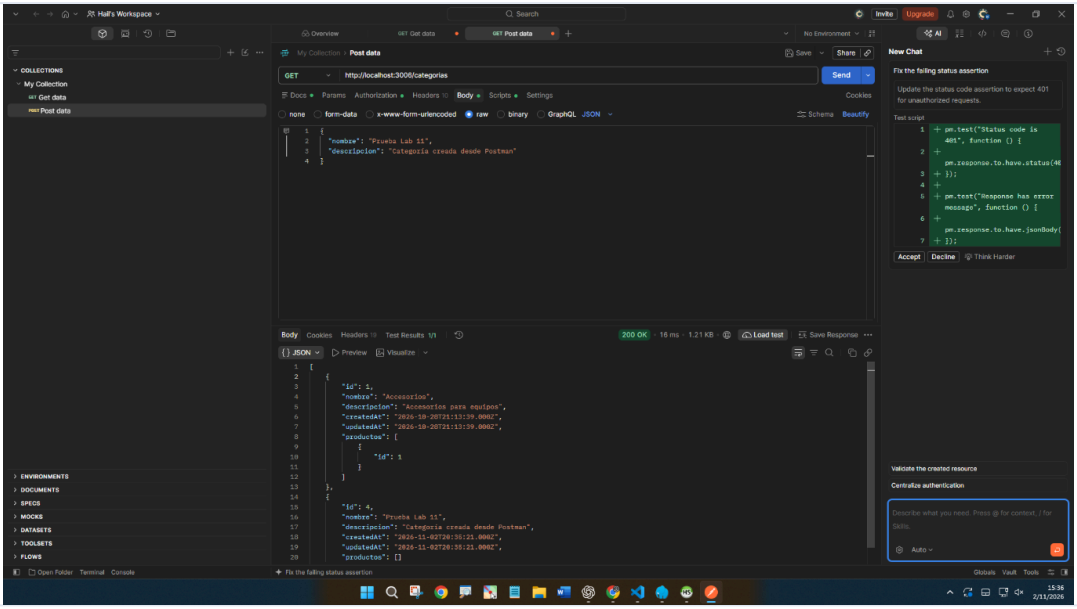
#	1 id	2 nombre	3 descripcion	4 createdAt	5 updatedAt
1	1	Accesorios	Accesorios para equipos	2026-10-28 21:13:39	2026-10-28 21:13:39
2	6	manzana	fruta	2026-11-02 21:04:25	2026-11-02 21:04:25

El correo y la consulta de usuarios visibles en el original se omitieron por no ser necesarios para esta evidencia.

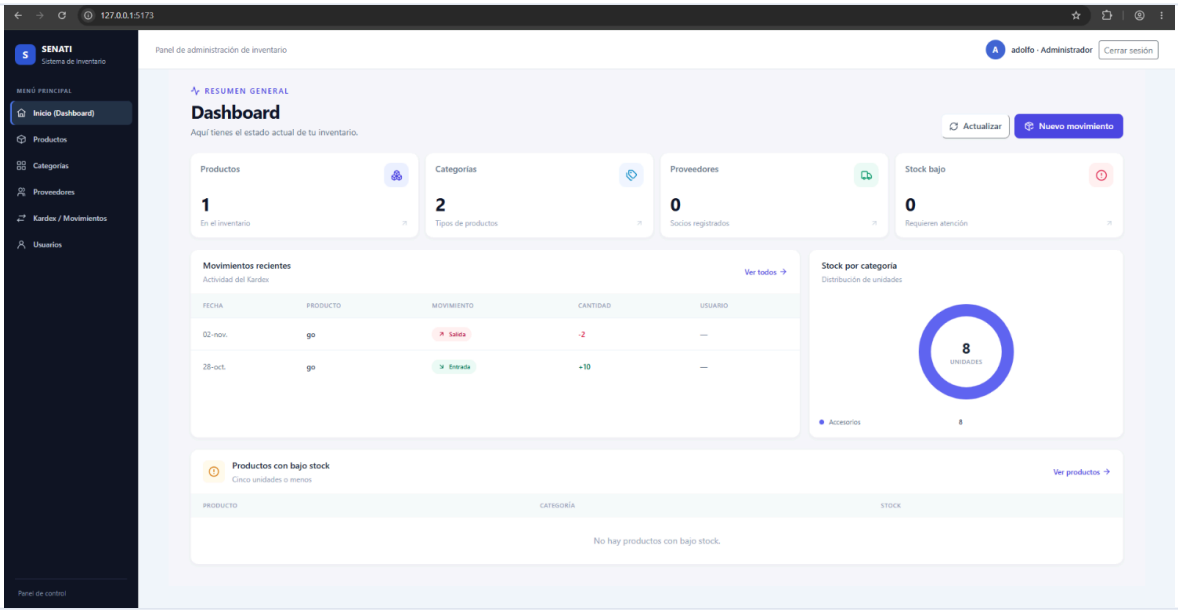
LABORATORIO 11 · ACLARACIÓN DE EVIDENCIA

Uso del JWT en la sesión y las pruebas

Las capturas del informe permiten comprobar el uso del token sin revelar el valor de la credencial. En el backend, el middleware de autenticación se instala antes de las rutas privadas; las peticiones a la API de categorías también pasan por esa validación.



Prueba GET de categorías en Postman: **200 OK**. El endpoint está detrás del middleware JWT, por lo que recibir esta respuesta confirma que la petición se atendió con una sesión válida.



Panel de inventario abierto como **Administrador**: evidencia de una sesión iniciada en la interfaz. Axios adjunta el token almacenado a las solicitudes de la API.

> **Conclusión:** el JWT sí se utilizó en el flujo del Laboratorio 11: la interfaz conserva la sesión y el GET protegido responde correctamente. El token se mantiene oculto porque es una credencial; ocultar su valor no oculta el resultado ni la evidencia de autenticación.